

REAL-TIME CYBER ATTACK DETECTION USING MAMBA (STATE SPACE MODELS)

Review 1

Submitted by

Name of Member 1: Soumaditya Mukherjee
Registration Number: 25BCE1118

Name of Member 2: Ramsha Riyasat Fatima
Registration Number: 25BCE1117

Name of Member 3: Kaivalya Lehekar
Registration Number: 25BCE1431

Department of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology, Chennai.
2026

1. Introduction

Servers get attacked every day. Someone sends a flood of fake traffic to crash a website, someone tries to sneak malware in through an open port, or an attacker quietly plants a backdoor to steal data later. Most small and mid-sized setups still depend on simple rule-based firewalls or slow, manually updated blocklists to catch this, which means a lot of attacks are noticed only after the damage is already done.

This project builds a real-time Network Intrusion Detection System, essentially a digital security guard, that watches the stream of incoming network traffic and predicts whether an attack is happening as it happens, not after the server has already gone down. The core of the system is Mamba, a State Space Model architecture that has recently emerged as a faster and lighter alternative to Transformers for processing long sequences of data. Network traffic is naturally sequential (packet after packet, connection after connection) which makes it a good fit for a sequence model like Mamba.

We are keeping the project fully open-source and budget-friendly. The dataset comes from Kaggle (UNSW-NB15, a widely used cybersecurity benchmark), and the model is built using existing, well-tested Python libraries such as mamba-ssm and PyTorch rather than writing the underlying state space math from scratch. This report covers the problem we are solving, why it matters, our proposed system design, the roadmap we plan to follow, and the early research we have looked at to justify our approach.

2. Real-World Problem and Justification :

2.1 The Problem

Cyber attacks such as Distributed Denial of Service (DDoS) floods, malware transmission, and backdoor intrusions are getting faster and harder to catch. Traditional intrusion detection tools generally fall into two camps. Signature-based systems only catch attacks they already have a rule for, so anything new slips through. Classic machine learning models (like simple decision trees or shallow neural networks) can spot new patterns but usually look at short windows of traffic and lose the bigger picture. Transformer-based deep learning models can look at longer patterns, but they are heavy, slow, and need a lot of memory and compute, which makes them impractical to run continuously on a live server that is already under attack.

In short, servers need a detector that is smart enough to spot unfamiliar attacks, fast enough to react in real time, and light enough to run without becoming a burden itself.

2.2 Why This Matters Now

Network traffic volumes keep growing, and attacks like DDoS floods are getting bigger and more automated every year. At the same time, a new class of AI models, State Space Models like Mamba, has shown that it is possible to process very long sequences of data (thousands of packets or events) in linear time instead of the quadratic time that Transformers need. This is a genuinely new opportunity: for the first time, a model that understands long traffic history can realistically run in real time on modest hardware. Applying this bleeding-edge architecture to cybersecurity, a field that badly needs faster detection, is a timely and practical direction.

2.3 Justification

We chose Mamba over traditional Machine Learning and over Transformers for three straightforward reasons. First, network intrusion detection is a sequence problem: an attack usually reveals itself as a pattern across many packets or connections over time, not in a single data point, so a model built for sequences fits naturally. Second, Mamba processes sequences in linear time and with low memory usage, which means it can realistically run live on a server without slowing it down, something a full Transformer struggles with. Third, everything we need is already available as free, open-source, and well-documented tools (the mamba-ssm library, PyTorch, and the UNSW-NB15 dataset on Kaggle), so the project is achievable within a student budget and timeline while still using genuinely current, research-grade technology.

3. Objectives

1. Build a working Network Intrusion Detection System that classifies incoming network traffic as normal or malicious (including specific attack types such as DDoS, malware-related traffic, and backdoor activity) using the UNSW-NB15 dataset.
2. Design and train a Mamba-based (State Space Model) deep learning model for this classification task, using the mamba-ssm and PyTorch libraries.
3. Develop a backend service using FastAPI that can accept traffic data, run it through the trained model, and return predictions in real time, along with a script that simulates live network traffic for testing and demonstration.
4. Build a dark-mode web dashboard that visualizes live traffic, flags suspicious activity clearly (visually flashing during an attack), and automatically blocks malicious IP addresses.
5. Compare the Mamba-based approach against traditional Machine Learning baselines and, where feasible, a Transformer baseline, in terms of detection accuracy, speed, and memory usage.

4. Scope

This project covers the full pipeline needed to demonstrate real-time intrusion detection: cleaning and preparing the UNSW-NB15 dataset, training and evaluating a Mamba-based detection model, building a FastAPI backend that serves predictions and simulates live traffic, and building a web dashboard that visualizes traffic and reacts to attacks. It also includes basic benchmarking of the model against traditional approaches on standard metrics like accuracy, false positive rate, detection latency, and memory usage.

We are not building a custom kernel-level packet capture system or any low-level networking driver. We are also not designing a new foundational AI architecture from scratch; we are applying the existing, published Mamba architecture (through the mamba-ssm library) to a new problem area rather than inventing a new model type. Deployment on live, production internet infrastructure is outside the scope of this academic project; our simulated traffic environment is meant to convincingly demonstrate real-time behavior rather than replace a production security appliance. Similarly, we are not covering physical network security, encrypted payload inspection, or attacks that occur entirely outside network traffic (such as social engineering or physical access breaches).

5. Feature Overview and Value Proposition

The system is built around a small set of clear, demonstrable features:

- Real-time traffic classification: incoming traffic is scored as normal or malicious within milliseconds, using the trained Mamba model.
- Multi-attack detection: the model is trained to recognize multiple attack categories present in UNSW-NB15, including DDoS-style floods, malware-related traffic, and backdoor activity, not just a single binary label.
- Live traffic simulation: a Python script generates a realistic stream of normal and malicious traffic so the whole pipeline can be demonstrated without needing an actual live attack.
- Command-center dashboard: a dark-mode web interface shows live traffic graphs, flashes red when an attack is detected, and lists the flagged IP addresses.
- Automatic IP blocking: when the model is confident traffic is malicious, the offending IP address is automatically added to a block list and reflected on the dashboard.

The value this brings is speed and efficiency. Because Mamba scales linearly with sequence length instead of quadratically like a Transformer, it can look at a longer history of traffic while

still running fast and light, which means quicker detection with fewer false alarms and less hardware cost than a heavier deep learning approach.

6. System Architecture

The proposed architecture is organized into four conceptual layers. The exact tools may shift slightly as we progress past this first review, but the overall shape of the system is expected to stay the same.

Layer	Purpose	Proposed Tools
Data Layer	Collects, cleans, and formats network traffic data (from the UNSW-NB15 dataset and the live traffic simulator) into sequences the model can read.	Python, Pandas, NumPy, Kaggle UNSW-NB15 dataset
AI Layer	The trained Mamba (State Space Model) that takes in a sequence of traffic features and predicts whether it is normal or a specific type of attack.	PyTorch, mamba-ssm
Backend API	Serves the model's predictions, manages the block list of flagged IPs, and streams simulated live traffic to the frontend.	FastAPI, Python, WebSockets
Frontend Dashboard	A dark-mode command center that displays live traffic graphs, visually flags attacks, and shows currently blocked IPs.	React or plain JavaScript/HTML/CSS, chart library

Data flows in one direction for the demonstration: the traffic simulator (or the offline dataset during training) feeds into the Data Layer, which prepares sequences for the AI Layer. The AI Layer's predictions are sent to the Backend API, which decides whether to flag or block an IP and pushes that update to the Frontend Dashboard in real time.

7. Proposed Methodology and Execution Roadmap

Our approach follows the standard applied machine learning pipeline, adapted for a sequence-based model and a live system. We plan to spread the work over roughly one year, moving through the following broad phases in order, with some overlap between the AI work and the backend/frontend work once the model is stable enough to be served.

Phase 1: Research and Data Preparation

We start by studying the Mamba architecture and how State Space Models work at a conceptual level, alongside a close look at the UNSW-NB15 dataset: its features, its attack categories, and its class imbalance. This phase involves cleaning the data, handling missing or inconsistent

values, encoding categorical features, normalizing numerical ones, and structuring the traffic into ordered sequences suitable for a sequence model.

Phase 2: Model Design and Training

Using the mamba-ssm library and PyTorch, we build the Mamba-based classifier, starting with a smaller version to validate that the pipeline works end to end before scaling up. We train the model on the prepared UNSW-NB15 sequences, tune hyperparameters, and evaluate it against a held-out test set. In parallel, we train baseline models (a traditional Machine Learning model such as Random Forest or XGBoost, and if time allows a lightweight Transformer) so we have something concrete to compare Mamba against.

Phase 3: Backend Development

With a working model in hand, we build the FastAPI backend: an endpoint that accepts a traffic sequence and returns a prediction, a live traffic simulator script that generates realistic normal and attack traffic, and the logic that manages the automatic IP block list based on model confidence.

Phase 4: Frontend and Integration

We then build the dark-mode dashboard and connect it to the backend, so live predictions flow through to real-time graphs, attack alerts, and the blocked-IP list. This phase focuses on making sure the whole pipeline works together smoothly, from simulated packet to on-screen alert.

Phase 5: Benchmarking, Refinement, and Documentation

In the final phase we benchmark the full system against the success criteria defined in Section 9, fix issues that come up, and refine both the model and the interface based on what the benchmarking shows. We close the project by documenting our findings, our architecture decisions, and the comparison between Mamba and the baseline approaches.

8. Preliminary Literature Survey

Before starting, we looked at a few key papers to make sure our approach is grounded in real, existing research rather than just a hunch.

The first is the original Mamba paper by Gu and Dao (2023), titled "Mamba: Linear-Time Sequence Modeling with Selective State Spaces." In simple terms, this paper introduces a new way to process sequences (like text, audio, or in our case, network traffic) that avoids the biggest weakness of Transformers. Transformers get slower and slower, in a quadratic way, as the sequence gets longer, because every part of the sequence has to be compared with every other

part. Mamba instead uses a state space model with a selection mechanism, meaning it can decide what information to keep or forget as it reads through a sequence, similar to how a person skims a long document but remembers the important parts. This lets Mamba process much longer sequences in linear time, using much less memory, while still keeping strong accuracy. This is exactly the property we need: network traffic over time is a long sequence, and we want to process it quickly without a huge memory footprint.

The second useful reference is a 2026 paper in the journal *Sensors* titled "NIDS-Mamba: Lightweight Network Intrusion Detection for IoT Sensor Networks via State Space Models." This paper does something very close to what we are proposing: it builds a Mamba-based intrusion detection system and tests it directly on network flow datasets, including a version of the UNSW-NB15 dataset. In plain language, the authors show that a Mamba-based model can reach very high accuracy while staying lightweight enough for devices with limited computing power, and that it handles the problem of imbalanced attack data (where normal traffic vastly outnumbers attack traffic) better than several baseline models. This paper is important evidence that our core idea, using Mamba for intrusion detection, is not just theoretically promising but has already shown strong practical results in closely related work.

Together, these two references support our project from two angles. The Mamba paper explains why the architecture is fast and efficient in general, and the NIDS-Mamba paper shows that this efficiency and accuracy carries over specifically into network intrusion detection. This gives us confidence that building our own Mamba-based NIDS on the UNSW-NB15 dataset is a reasonable and well-grounded direction for this project.

9. Expected Outcomes

- A cleaned and properly structured version of the UNSW-NB15 dataset, ready for sequence-based model training.
- A trained Mamba-based intrusion detection model capable of classifying traffic as normal or as a specific attack type (DDoS, malware, or backdoor).
- A working FastAPI backend that can serve real-time predictions and simulate live network traffic for demonstration.
- A functional dark-mode dashboard that visualizes traffic, flags attacks, and automatically blocks malicious IPs.
- A comparison report showing how the Mamba-based model performs against traditional Machine Learning and, if included, a Transformer baseline, in terms of accuracy, speed, and memory usage.

10. Benchmarking and Success Criteria

We will judge the success of the system against a small set of measurable, practical criteria rather than vague goals:

Criterion	What We Are Measuring	Target
Detection Speed	Time taken to classify a traffic sequence and return a prediction	Prediction returned in real time, well under one second per sequence
Detection Accuracy	Overall accuracy and F1-score on the UNSW-NB15 test set	Accuracy and F1-score comparable to or better than the traditional Machine Learning baseline
False Positive Rate	How often normal traffic is wrongly flagged as an attack	Kept as low as possible, since frequent false alarms make the system unusable in practice
Memory Efficiency	Peak memory used during inference, compared against a Transformer baseline	Noticeably lower memory usage than an equivalent Transformer-based model
End-to-End Responsiveness	Delay between simulated attack traffic appearing and the dashboard flagging or blocking it	Alert and auto-block reflected on the dashboard within a few seconds of the attack starting

11. Key Risks and Mitigations

Risk	Mitigation
The UNSW-NB15 dataset is imbalanced, with far more normal traffic than attack traffic, which can bias the model.	Use resampling techniques and weighted loss functions during training, and evaluate using F1-score and confusion matrices rather than accuracy alone.
The mamba-ssm library is relatively new and may have installation or compatibility issues, especially around GPU support.	Start integration and environment setup early, keep a traditional Machine Learning model as a fallback baseline, and document setup steps carefully as we go.
Real-time performance in the demo environment may not fully reflect real production network conditions.	Be transparent in the report that our simulated traffic environment is meant to demonstrate the pipeline's behavior, not replace real-world production testing.
Team members may have uneven experience across the three specializations (AI, backend, frontend).	Keep the workflow split by strength as proposed, hold regular sync-ups between the AI, backend, and frontend work, and document interfaces (like the API format) clearly so the three parts integrate smoothly.

Risk	Mitigation
Scope creep, trying to add too many extra features could delay the core detection pipeline.	Keep to the defined scope in Section 3, treat any extra features as optional stretch goals only after the core system works end to end.